An Orthogonal Functional Network for Function Approximation

Yongquan Zhou, Xueyan Lu, Zhucheng Xie and Bai Liu

College of Mathematics and Computer Science
Guangxi University for Nationalities
Nanning, Guangxi 530006, CHINA
yongquanzhou@126.com

Abstract

In this paper, a novel orthogonal functional network is proposed, it is a recently introduced extensive of functional networks. A learning algorithm of orthogonal functional network is proposed, the learning of functional parameters use Lagrange multipliers by means of auxiliary function and solving a system of linear equations obtain parameters. An experiment in approximating functions is given. The results show that the learning algorithm presented in the paper has excellent performance in approximation error.

Keywords: *functional network; orthogonal functional network; functional parameters; learning algorithm; function approximation.*

1. Introduction

Castillo E introduced the functional network in 1998. It is an extension of the standard neural network. Unlike neural networks, it deals with general functional models instead of sigmoid-like ones, and in these networks there are no weights associated with the links connecting neurons. Therefore, due to functional network have many advantages, the problem that can be solved by the neural network also can be formulated by functional network [1].There are also many examples that can not be solved by the neural network but can [1-3] be naturally formulated using the functional network. Its main advantage is that the domain knowledge and data knowledge are involved together. Functional network deals with the general functional model instead of the sigmoid-like one. The initial topology of functional network must be decided by the functional model, says, the functional equation. Sometimes, the structure of the functional network can be simplified by its model (i.e. equation) ahead of resolving problem. In the functional network, the weights among the neurons of the different layers are removed. Therefore, the connection only shows the information flow along the neurons. Every neuron's active function can be freely selected in a given dictionary, and the functional network learns to modify the neuron function while the traditional one modifies its weight. At the same time, the neuron function may be multivariate, multi-argument, and different instead of univariate, single-argument, and identical in standard neural network. Thus, this method is better than the standard neural network.

Functional network is a very useful general framework for solving a wide range of problems: The solving of differential functional and difference equation [2], nonlinear time series and prediction modeling[3], factorization model of multivariate polynomials [4], the identification of nonlinear system [5], CAD, linear and nonlinear regression[4], etc. Some of these applications have been developed in [5-9].

In this paper, an orthogonal functional network model is presented whose neuron functions are approximated by orthogonal basis functions. The well-known functions such as Fourier series, Bessel function, Legendre polynomials, etc. are all orthogonal functions. The parameters of the orthogonal functional network are determined by solving a series of equations and used in the learning algorithm. It has been proved that the training algorithm will always converge and find optimum parameters. The results show that the learning algorithm presented in the paper has excellent performance in approximation error.

The rest of the paper outlines as follows. In section 2, the elements of functional network are introduced. The orthogonal function is introduction in section 3. The model of an orthogonal functional neuron and orthogonal functional network model are given in section 4. The learning algorithm of orthogonal functional network is presented in section 5 .Taking the typical Hénon mapping in the time series, and approximation of continuous function as examples show the method in section 6.Finally, section 7 is concludes this paper.

978-0-7695-3357-5/08 $25.00 © 2008 IEEE
DOI 10.1109/ICICTA.2008.164

2. Functional network

A functional network [5] consists of the following elements (see Figure 1):

1). *A layer of input storing units:* This first layer contains the input data. Input units are represented by small black circles with its corresponding name (x_1, x_2, and x_3 in Figure 1)

2). *A layer of output units:* This is the last layer, and contains the output data. Output units are also represented by small black circles with its corresponding name (x_6 in Figure 1)

3). *One or several layers of neurons or computing units:* A neuron is a computing unit which evaluates a set of input values, coming from the previous layer of neurons or input units, and gives a set of output values to the next layer, of neurons or output units. The computing units are connected to each other, in the sense that output from one unit can serve as part of input to another neuron or to the units in the output layer. Once the input values are given, the output is determined by the neuron type, which can be defined by a function. For example, assume that we have a neuron with s inputs $(x_1, x_2, ..., x_s)$ and k outputs $(y_1, y_2, ..., y_k)$, then, we assume that there exist k functions f_j ; $j = 1,2,..,k.$ such that

$$y_j = f_j(x_1, x_2, .., x_s)\,;\, j = 1,2,..,k.$$

The functions f_j are not arbitrary, but determined by the structure of the network, as we shall see later. Neurons are represented by circles with the name of the corresponding f_j function inside (f_1, f_2 and f_3 in Figure 1).

4). *A set of directed links:* They connect the input layer to the first layer of neurons, neurons of one layer to neurons of next layer, and the last layer of neurons to the output units, connections are represented by arrows, indicating the information flow direction.

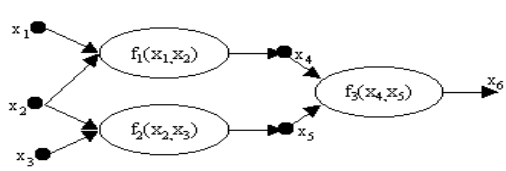

Figure 1. A functional network model

3. Orthogonal function

According to the theory of orthogonal functions [10], [11], an arbitrary function $f(x), f:[a,b] \to R$ .will have an orthogonal polynomial:

$$F_n(x) = a_1\phi_1(x) + a_2\phi_2(x) + a_3\phi_3(x) + \cdots + a_n\phi_n(x) \quad (1)$$

Such that

$$\lim_{n\to\infty}\int_a^b (f(x) - F_n(x))^2 dx = 0 \quad (2)$$

Where

$$\int_a^b \phi_i(x)\phi_j(x)dx = \begin{cases} 0 & i \neq j \\ A_i & i = j \end{cases}$$

$$a_i = \frac{\int_a^b f(x)\phi_i(x)dx}{A_i}, \quad i = 1,2,... \quad (3)$$

$\{\phi_1(x), \phi_2(x), \cdots \phi_n(x)\}$ is an orthogonal basis functions set. The above equations show that a function can be approximated by an orthogonal function set and coefficient a_i is unique. Equation (3) shows that the coefficients a_i, will not be available from the integration if function $f(x)$ is unknown. Unfortunately, the models of most dynamic systems are unknown or hard to determine. Therefore, it is necessary to develop a method that can use the input and output data of the systems to determine the coefficient a_i.

The above discussion is mainly for a single-variable function $f(x)$. As for multi-variable functions, a multi-variable orthogonal function set can be generated by integrating several single-variable orthogonal function sets. For example, there is a function with m variables. Its corresponding m -variable orthogonal function set will be $\{\Phi_1(X), \Phi_2(X), ...\}$.Each of the orthogonal functions is defined as

$$\Phi_i(X) = \phi_{1i}(x_1)\phi_{2i}(x_2)...\phi_{mi}(x_m) \quad (4)$$

There are several orthogonal functions such as Fourier series, Bessel Function, and Legendre Polynomials. Each of them has its application domain in function approximation. For example, Fourier series is suited to approximate sine/cosine functions. However, it does not have the property of completeness. In other words, there is no guarantee that the approximation of functions will be converged.

4. Orthogonal functional network

One simple of an orthogonal functional network is given in Figure 2. Where the input layer consists of the unit X , the first layer of network contains the orthogonal basis functions { $\phi_i, i = 1,2,...,n.$ }, the output layer consists of the unit y.

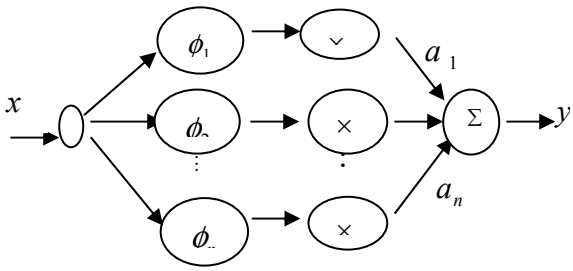

Figure 2. A simple orthogonal function network

Note the following difference between orthogonal neural networks [12] and orthogonal functional networks:

1). In orthogonal neural networks function ϕ_i are given, and some weights associated with the links or connections have to be learned. In orthogonal functional networks there are no weights, and the ϕ_i function must be learned. In addition, the inputs and outputs can be real numbers.

2).Unlike orthogonal neural networks, where output units are the output of a single neuron (the last one), in orthogonal functional networks output units are the output of one or more neurons. This implies some compatibility conditions, i.e., the larger the number of connected neurons to the output units, the less the number of degrees of freedom.

The input of the orthogonal functional network (see Figure 2) is x , and the output is

$$y = f(x) = \sum_{i=1}^{n} a_i \phi_i(x) \tag{5}$$

The orthogonal neural network to be presented in this paper is based on orthogonal basis functions, whose is called orthogonal functional network.

5. Orthogonal functional network learning algorithm

The general orthogonal function network as shown in Figure 3.

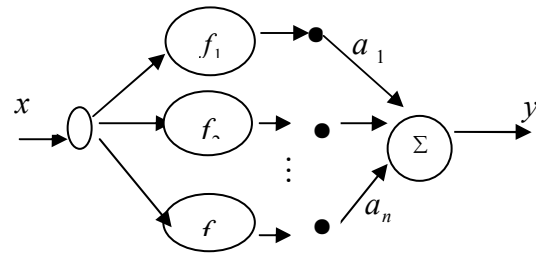

Figure 3. A general orthogonal function network

The suggests approximating each of the functions f_i in Figure 3 by considering a linear combination of known orthogonal basis functions. If we assume that $\{\phi_{ij}(x_j) \mid i = 1,2,\cdots,k;\ j = 1,2,\cdots,m\}$ is a set of linearly independent orthogonal basis functions, then, we have

$$y = \sum_{i=1}^{k} a_i f_i(x)$$

Where

$$f_i(x) = \sum_{j=1}^{m} a_{ij} \phi_{ij}(x)$$

Thus, the output of the orthogonal functional network (see Figure 3) is

$$y = \hat{f}(x) = \sum_{i=1}^{k} \sum_{j=1}^{m} a_{ij} \phi_{ij}(x) \tag{6}$$

where the $\{\phi_{ij}(x) \mid i = 1,2,...,k; j = 1,2,...,n\}$ is a set of given linearly independent orthogonal functions, capable of approximating $f(x)$ to the desired accuracy. To estimate the coefficients $\{a_{ij} \mid i = 1,2,\cdots k; j = 1,2,\cdots,m\}$, we given set of data pair's $(x_j, y_j), j = 1,2,...,m.$ .where $y_j = f(x_j)$. and define the error

$$\begin{aligned} e_p &= y_p - f(x_{jp}) \\ &= y_p - \sum_{i=1}^{k} \sum_{j=1}^{m} a_{ij} \phi_{ij}(x_{jp}) \end{aligned} \tag{7}$$

Thus, we minimize the sum of the square errors :

$$E_0 = \frac{1}{2}\sum_{p=1}^{P} e_p$$
$$= \frac{1}{2}\sum_{p=1}^{P}[y_p - \sum_{i=1}^{k}\sum_{j=1}^{m} a_{ij}\phi_{ij}(x_{jp})]^2 \qquad (8)$$

To guarantee uniqueness of representation we specify the values of the function f at a point, that is:

$$f(x_{j0}) = u_{j0}, j = 1,2,\cdots,m$$

where u_{j0} is the selected point. Then, the Lagrange multipliers technique leads to the auxiliary function:

$$E_\lambda = E_0 + \frac{\lambda}{2}\sum_{j=1}^{m}(f_{ij}(x_{j0}) - u_{j0})^2$$
$$= \sum_{p=1}^{P}[y_p - \sum_{i=1}^{k}\sum_{j=1}^{m} a_{ij}\phi_{ij}(x_{jp})]^2 + \frac{\lambda}{2}\sum_{j=1}^{m}(f_{ij}(x_{j0}) - u_{j0})^2 \qquad (9)$$

The minimum can be obtained by solving the following system of linear equations, where the Unknowns are the coefficients in the set $\{a_{ij} \mid i = 1,2,\cdots k; j = 1,2,\cdots,m\}$ and the multipliers λ :

$$\begin{cases} \dfrac{\partial E_\lambda}{\partial a_{tr}} = -\sum_{p=1}^{P} e_p[\sum_{t\neq i=1}^{k}\sum_{r\neq j=1}^{m} a_{ij}\phi_{ij}(x_j)]\phi_{tr}(x_r) + \lambda(f(x_{j0}) - u_{j0})\phi_{tr}(x_{j0}) = 0 \\ \dfrac{\partial E_\lambda}{\partial \lambda} = \dfrac{1}{2}\sum_{j=1}^{m}(f(x_{j0}) - u_{j0})^2 = 0 \end{cases} \qquad (10)$$

Then, solving this linear system of equations we get the optimal neuron functions for a give problem.

6. Experimental Results

In this section, we will apply the orthogonal basis functions functional network developed above to approximate functions and discuss their results. To estimate the performance of approximate the root mean square error *(RMSE)* defined as:

$$RMSE = \sqrt{\frac{1}{r}\sum_{p=1}^{r}\| y_p - \hat{y}_p \|^2} \qquad (11)$$

Where $\hat{y}_p$ the network output and the norm function are $\| \bullet \|$ reduces to the usual absolute values function $| \bullet |$.

Example 1: The Hénon map [5] is one of the most popular dynamical systems exhibiting chaos, and can be defined in the following way:

$$x_n = 1 - 1.4x_{n-1}^2 + 0.3x_{n-2} \qquad (12)$$

A time series plot of the Hénon map for the initial conditions $x_0 = 0.5, x_1 = 0.5$ is given in Figure 4.

This figure shows the seemingly stochastic dynamics of the system that is due to the sensitivity of the initial conditions characteristic of chaotic maps.

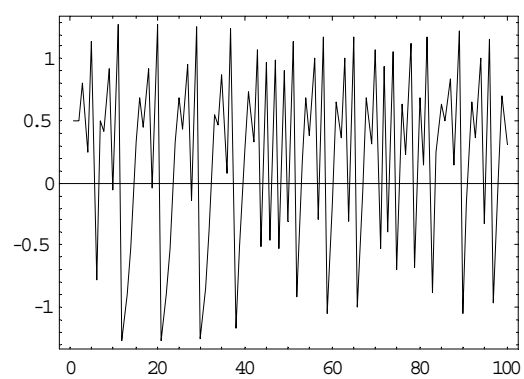

Figure 4. Time series for the Hénon map

Suppose that we have the time series consisting of 100 points shown in Figure 4 and that we want to obtain a representative model of the underlying dynamics. We can use the functional network in Figure 5, where, f_1, f_2 and f_3^{-1} are orthogonal functional neural neuron functions.

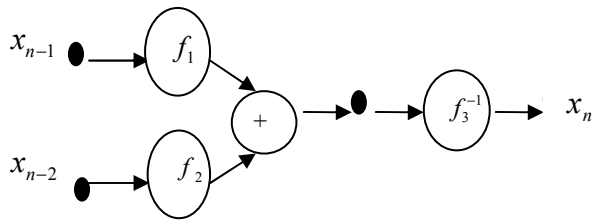

Figure 5. A five-layer orthogonal functional network associated with the Hénon map

Table 1. The results of Castillo model

Model	$\phi_k \quad k=1,2$	ϕ_3	RMSE
1	$\{1, x, x^2\}$		0
2	$\{\sin x, \sin 2x, \cos x, \cos 2x\}$	$\{1, x\}$	0.005932
3	$\{\sin x, \sin 2x, \cos x, \cos 2x\}$		0.003378
4	$\{1, \log(2+x), \log(3+x), \log(4+x), \log(5+x)\}$		0.00001598

Table 2. The results of orthogonal functional network model

Model	$\phi_k \quad k=1,2$	RMSE
1	Legendre polynomials basis: $\{1, x, \frac{1}{2}(3x^2-1), \frac{1}{2}(5x^3-3x)\}$	0
2	Chebyshev polynomials basis: $\{1, x, 2x^2-1, 4x^3-3x\}$	0
3	Fourier Series: $\{1, \sin x, \cos x, \sin 2x, \cos 2x\}$	0.23×10^{-6}
4	Fourier Series: $\{1, \sin x, \cos x, \sin 2x, \cos 2x, \sin 3x, \cos 3x\}$	1.90767×10^{-7}

Compare to Castillo E model(see Table 1), simulation results show that the orthogonal functional network learning algorithm used in the time series analysis has high precision (see Table 2).

7. Conclusions

This paper presents an orthogonal functional network, whose functional neuron is approximated by orthogonal basis function and the parameters of orthogonal functional network are determined by solving a series of linear equations. Meanwhile, the learning algorithm based on minimizing a sum of squares with a unique minimum has been proposed for the purpose of approximating function. Experimental results show that this learning algorithm is effective and practical.

Acknowledgements

This work is supported by Grants 60461001 from NSF of China and the project Supported by Grants 0542048, 0832082 from Guangxi Science Foundation.

References

[1] Castillo E. "Functional Networks", *Neural Processing. Letters*, 7, pp. 151-159, 1998.

[2] Castillo E, Cobo A, Gutierrez J M, Pruneda E. "Working with Differential, Functional and Difference Equations Using Functional Networks", *Appl Math Model,* 23, pp.89-107,1999.

[3] Castillo E, Gutierrez J M. "Nonlinear Time Series Modeling and Prediction Using Functional Networks, Extracting Information Masked by Chaos", *Phys. Lett. A,*244, pp.71-84, 1998.

[4] Zhou Yong-quan. "An approximate factorization model of multivariate polynomials based on algebra neural networks and learning algorithm". *Journal of Computation Research & Development*, .36(6), pp.668-674, 1999. (In Chinese)

[5] Castillo, E, Cobo A, Gutierrez, J M and Pruneda, R. E. *An Introduction to Functional Networks with Applications*, Kluwer Academic Publishers: New York, 1998.

[6] Castillo, E., Gutierrez, J. M., Hadi, A. S. and Lacruz, B. "Some Applications of Functional Networks in Statistics and Engineering", *Technometrics*, 43(1), pp.10-24, 2001

[7] Castillo, E., Hadi, A. S. and Lacruz, B. "Optimal Transformations in Multiple Linear Regression Using Functional Networks", *Lecture Notes in Computer Science*, 2084, Part I, pp. 316-324, 2001

[8] Castillo, E., Hadi, A. S., Lacruz, B. and Pruneda, R. E. "Semi-Parametric Nonlinear Regression and Transformation Using Functional Networks", Sent for publication, 2001.

[9] Castillo E, A. Cobo, J. Gutierrez, and E.Pruneda, *Introduction to Functional Networks with Applications: A neural Based Paradigm.* Norwell, MA: Kluwer, 1998

[10] R.Courant and D. Hilbert, *Methods of Mathematical Physics.* New York: inter science Publishers, 1995.

[11] F. B. Hildebtand. *Advanced Calculus for Applications. Englewood Cliffs,* NJ: Prentice-Hall, 1976.

[12] Shiow-Shung Yang and Ching-.Shiow Tseng, "An Orthogonal Neural Network for Function Approximation," IEEE *Trans. on Systems, Man, and Cybernetics,*26(10),pp.779-785,1996.